Analysis Framework

Session No.: 04
Course Name: Design & Analysis of Algorithms
Course Code:
Instructor Name: Dr. Mili Dhar
Duration: 50
Date of Conduction of Class: 11-03-2025

Recap

- Asymptotic Notations
- Big O
- Big Omega
- Theta
- Small o
- Small omega

Learning Outcome

By the end of this session, you will be able to:

- analyze any algorithm on varying input sizes.

Session Outline

- Measuring an input sizes
- Units of measuring running time
- Orders of Growth
- Case Efficiencies: Worst Case, Average Case, Best Case

Let's continue with the starting question of the session.

Scenario 1: Prepare tea for 1 person.

Scenario 2: Prepare tea for 100 persons.

Scenario 3: Prepare tea for 1000 persons.

Analysis Framework and Empirical Analysis

- The Analysis Framework is a structured method to evaluate the efficiency of algorithms using theoretical and empirical approaches. Let's explore this with detailed steps and examples for better understanding. Empirical analysis evaluates an algorithm's actual performance through experimentation rather than theoretical calculations.

Steps in the Analysis Framework

- **Problem Definition**
- **Identify Basic Operations**
- **Complexity Analysis (Time & Space)**
- **Best, Worst, and Average Case**

Problem Definition

- Clearly state the problem, input size, and desired output.
- Example:

  Problem: Linear Search for a Key.

  Input: Array of integers, arr[] with size n.

  Output: Find an element in the array.

Identify Basic Operations

The basic operation is the key step that determines the algorithm's complexity.

- Example:
- Algorithm: Linear Search (to find the element)
- Basic Operation: Comparing elements (comparison step).

Complexity Analysis

Time Complexity Analysis

- **Best Case:** Minimum steps needed.
- **Worst Case:** Maximum steps needed.
- **Average Case:** Expected steps based on random inputs.

Space Complexity Analysis

- Tracks additional memory usage during algorithm execution.

Activity 1

Activity 1

```
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i; // Element found at index i
        }
    }
    return -1; // Element not found
}
```

Best Case: O(1) — Key found at the first position.
Worst Case: O(n) — Key found at the last position or absent.
Average Case: O(n) — For random data.

Activity 2

```
int binarySearch(int arr[], int n, int key) {
    int left = 0, right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid; // Element found at index mid
        }
        else if (arr[mid] < key) {
            left = mid + 1; // Search in the right half
        }
        else {
            right = mid - 1; // Search in the left half
        }
    }

    return -1; // Element not found
}
```

Time Complexity Analysis

- Best Case: $O(1)$ – Key found at the middle in the first step.
- Worst Case: $O(\log n)$ – Key is found at the extreme ends or not present.
- Average Case: $O(\log n)$ – For random data.

Summary

Theoretical Analysis shows that Binary Search is highly efficient with O(logn) complexity.
Empirical Analysis confirms this efficiency by testing it on real data sets.

Summary

- Important factors for an algorithm
  - **input size** (how your algorithm's running time grows when you increase input size)
  - **basic operation** (which statements in the algorithm mostly contribute to the running time)
- Orders of Growth
  - How algorithm's runtime increases when you increase the input size?
- Case Efficiencies: Worst, Average, Best

Post Session Activity

```java
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                // Swap arr[j] and arr[j+1]
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

You need to find the **input size** and **basic operation** (mostly contributing to the running time of the problem).

Post Session Assessment

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
REOILF